

Improving diagnostics for sender expressions

Document:

P3164R1

Authors:

[Eric Niebler](#)

Date:

July 14, 2024

Source:

[Github](#)

Issue tracking:

[Github](#)

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Audience:

Library Evolution Working Group

- [Synopsis](#)
- [Executive Summary](#)
- [Revision History](#)
- [Improving early diagnostics](#)
 - [Problem Description](#)
 - [Non-dependent senders](#)
 - [Suggested Solution](#)
 - [Comparison Table](#)
 - [Design Considerations](#)
- [Improving late diagnostics](#)
 - [Problem description](#)
 - [Suggested solution](#)
 - [Another problem and a solution](#)
 - [Comparison Table](#)
- [Proposed Wording](#)
- [Acknowledgments](#)

Synopsis

This paper aims to improve the user experience of the sender framework by giving it better diagnostics when used incorrectly.

First, it moves the diagnosis of many invalid sender expression earlier, when the expression is constructed, rather than later when it is connected to a receiver. A trivial change to the sender adaptor algorithms makes it possible for the majority of sender expressions to be type-checked this way, giving the user immediate feedback when they've made a mistake.

Second, this paper proposed changes to the `transform_completion_signatures` alias template to allow it to serve as a meta-exception propagation channel. This helps with late (connect-time) type checking by allowing type computation errors from deeply nested senders to propagate to the API boundary, where they can be reported concisely.

Executive Summary

Below are the specific changes this paper proposes in order to improve the diagnostics emitted by sender-based codes:

1. Define a "non-dependent sender" to be one whose completions are knowable without an environment.
2. Extend the awaitable helper concepts to support querying a type whether it is awaitable in an arbitrary coroutine (without knowing the promise type). For example, anything that implements the awaiter interface (`await_ready`, `await_suspend`, `await_resume`) is awaitable in any coroutine, and should function as a non-dependent sender.
3. Add support for calling `get_completion_signatures` without an environment argument.
4. Change the definition of the `completion_signatures_of_t` alias template to support querying a sender's non-dependent signatures, if such exist.
5. Require the sender adaptor algorithms to preserve the "non-dependent sender" property wherever possible.

6. Add "Mandates:" paragraphs to the sender adaptor algorithms to require them to hard-error when passed non-dependent senders that fail type-checking.
7. Extend the eager type checking of the `let_` family of algorithms to hard-error if the user passes a lambda that does not return a sender type.
8. Change `transform_completion_signatures` to propagate any intermediate types that are not specializations of the `completion_signatures<>` class template. For type errors that occur when computing a sender's completion signatures, sender authors can return a custom type that describes the error and have it automatically propagate through adaptors that use `transform_completion_signatures`.
9. For any algorithm that eagerly connects a sender (`sync_wait`, `start_detached`, `ensure_started`, `split`), hard-error (*i.e.* `static_assert`) if the sender fails to type-check rather than SFINAE-ing the overload away.

Revision History

R1:

- Change the specification of `transform_completion_signatures` to propagate types that are not specialization of the `completion_signatures<>` class template. This makes it easier to use an algorithm's completion signatures to communicate type errors from child senders.
- For the customization points `let_value`, `let_error`, and `let_stopped`, mandate that the callable's possible return types all satisfy `sender`.
- Change *Requires:* to *Mandates:* for algorithms that eagerly connect senders.

R0:

- Original revision

Improving early diagnostics

Problem Description

Type-checking a sender expression involves computing its completion signatures. In the general case, a sender's completion signatures may depend on the receiver's execution environment. For example, the sender:

```
read_env(get_stop_token)
```

... when connected to a receiver `rcvr` and started, will fetch the stop token from the receiver's environment and then pass it back to the receiver, as follows:

```
auto st = get_stop_token(get_env(rcvr));
set_value(move(rcvr), move(st));
```

Without an execution environment, the sender `read_env(get_stop_token)` doesn't know how it will complete.

The type of the environment is known rather late, when the sender is connected to a receiver. This is often far from where the sender expression was constructed. If there are type errors in a sender expression, those errors will be diagnosed far from where the error was made, which makes it harder to know the source of the problem.

It would be far preferable to issue diagnostics while *constructing* the sender rather than waiting until it is connected to a receiver.

Non-dependent senders

The majority of senders have completions that don't depend on the receiver's environment. Consider `just(42)` -- it will complete with the integer 42 no matter what receiver it is connected to. If a so-called "non-dependent" sender advertised itself as such, then sender algorithms could eagerly type-check the non-dependent senders they are passed, giving immediate feedback to the developer.

For example, this expression should be immediately rejected:

```
just(42) | then([](int* p) { return *p; })
```

The `then` algorithm can reject `just(42)` and the above lambda because the arguments don't match: an integer cannot be passed to a function expecting an `int*`. The `then` algorithm can do that type-checking only when it knows the input sender is non-dependent. It couldn't, for example, do any type-checking if the input sender were `read_env(get_stop_token)` instead of `just(42)`.

And in fact, some senders *do* advertise themselves as non-dependent, although P2300 does not currently do anything with that extra information. A sender can declare its completions signatures with a nested type alias, as follows:

```
template <class T>
struct just_sender {
    T value;

    using completion_signatures =
        std::execution::completion_signatures<
            std::execution::set_value_t(T)
        >;

    // ...
};
```

Senders whose completions depend on the execution environment cannot declare their completion signatures this way. Instead, they must define a `get_completion_signatures` customization that takes the environment as an argument.

We can use this extra bit of information to define a `non_dependent_sender` concept as follows:

```
template <class Sndr>
concept non_dependent_sender =
    sender<Sndr> &&
    requires {
        typename remove_cvref_t<Sndr::completion_signatures>;
    };
```

A sender algorithm can use this concept to conditionally dispatch to code that does eager type-checking.

Suggested Solution

The authors suggests that this notion of non-dependent senders be given fuller treatment in P2300. Conditionally defining the nested typedef in generic sender adaptors -- which may adapt either dependent or non-dependent senders -- is awkward and verbose. We suggest instead to support calling `get_completion_signatures` either with *or without* an execution environment. This makes it easier for authors of sender adaptors to preserve the "non-dependent" property of the senders it wraps.

We suggest that a similar change be made to the `completion_signatures_of_t` alias template. When instantiated with only a sender type, it should compute the non-dependent completion signatures, or be ill-formed.

Comparison Table

Consider the following code, which contains a type error:

```
auto work = just(42)
| then([](int* p) { // <<< ERROR here
    //...
});
```

The table below shows the result of compiling this code both before the proposed change and after:

Before	After
no error	error: static_assert failed due to requirement '_is_completion_signatures<ustdex::ERROR<ustdex::WHERE (ustdex::IN_ALGORITHM, ustdex::then_t), ustdex::WHAT (ustdex::FUNCTION_IS_NOT_CALLABLE), ustdex::WITH_FUNCTION ((lambda at hello.cpp:57:18)), ustdex::WITH_ARGUMENTS (int)>>' static_assert(_is_completion_signatures<completions>); ^ ~~~~~

This error was generated with with [ustdex](#) library and Clang-13.

Design Considerations

Why have two ways for non-dependent senders to publish their completion signatures?

The addition of support for a customization of `get_completion_signatures` that does not take an environment obviates the need to support the use of a nested `::completion_signatures` alias. In a class, this:

```
auto get_completion_signatures() ->
    std::execution::completion_signatures<
        std::execution::set_value_t(T)
    >;
```

... works just as well as this:

```
using completion_signatures =
    std::execution::completion_signatures<
        std::execution::set_value_t(T)
    >;
```

Without a doubt, we could simplify the design by dropping support for the latter. This paper suggests retaining it, though. For something like the `just_sender`, providing type metadata with an alias is more idiomatic and less surprising, in the author's opinion, than defining a function and putting the metadata in the return type. That is the case for keeping the `typename Sndr::completion_signatures` form.

The case for adding the `sndr.get_completion_signatures()` form is that it makes it simpler for sender adaptors such as `then_sender` to preserve the "non-dependent" property of the senders it adapts. For instance, one could define `then_sender` like:

```
template <class Sndr, class Fun>
struct then_sender {
    Sndr sndr_;
    Fun fun_;

    template <class... Env>
    auto get_completion_signatures(Env&&... env) const
        -> some-computed-type;

    //....
};
```

... and with this one member function support both dependent and non-dependent senders while preserving the "non-dependent-ness" of the adapted sender.

Improving late diagnostics

Problem description

Experience implementing and using sender-based libraries has taught the author several things:

1. Concepts-based constraints on sender algorithms and their inner workings do more harm than good. The diagnostics are generally poor. The constraint failure may happen deep in a sender expression tree, but the diagnostic the user sees is simply: "no overload found". That gives users exactly zero information about the cause of the error.

Dropping the one-and-only possible overload from the overload set typically doesn't help users either, most of whom don't care about SFINAE. What they want are good diagnostics.

2. The current specification of the customization points and utilities make type errors SFINAE-able: either a construct type-checks or else it is ill-formed. That makes it very difficult for sender adaptors to propagate type errors from their child senders. In runtime code, we have exceptions to propagate errors to API boundaries. We have no equivalent for type computations, and P2300's current facilities offer no help.

Suggested solution

To address the first issue, the author recommends using `static_asserts` instead of `requires` clauses for type errors in sender algorithms.

But what condition should we put in the `static_assert`? If we use the same predicates that are in the `requires` clauses, the errors will be little better. Instead of "no overload found", users will see: "`static_assert: sender_to<Sndr, Rcvr> evaluated to false`", followed by a (lengthy and probably truncated) concepts backtrace. Buried in there somewhere *may* be the cause of the error for those enterprising enough to dig for it.

This brings us to the second issue: propagating type errors from deep inside a sender tree evaluation to the API boundary where it can be concisely reported to the user.

The best way the author has found to report "late" (at connect time) type-checking failures is via the sender's completion signatures. If a type error happens while trying to compute `completion_signatures_of_t<Sndr, Env>`, instead of making the type ill-formed, it is better for it to

name a type that communicates the error to the user.

Algorithms like `sync_wait` can then `static_assert` that the result of `completion_signatures_of_t<Sndr, Env>` is a specialization of the `completion_signatures` class template. If it instead names a type that is descriptive of the error, the name of that type will appear prominently in the compiler's (blissfully short) diagnostic.

Consider the following code, which has a type error in it:

```
thread_context ctx; // non-standard extension
auto sch = ctx.get_scheduler();

using namespace std::execution;

auto work = read_env(get_delegatee_scheduler)
| let_value([](auto sched) {
    // create some work to delegate to the main thread.
    auto delegated_work =
        just() | then([] {
            std::puts("Hello, world!");
        });

    // launch the work on the delegation scheduler.
    start_on(sched, delegated_work); // <<< ERROR HERE
});

auto s = start_on(sch, work);

sync_wait(s);
```

The error in the code above is that the lambda passed to `let_value` must return a sender. The error cannot be caught early because this is a dependent sender: the type of the delegation scheduler isn't known until we pass the sender to `sync_wait`.

Compiling this with the [ustdex](#) library, which uses the suggested technique of propagating descriptive type errors via the completion signatures, results in the following diagnostic:

```
[build] /home/eniebler/Code/ustdex/include/ustdex/detail/sync_wait.hpp:139:7: error: static_assert failed due to requirement
'_is_completion_signatures<
ustdex::ERROR<ustdex::WHERE (ustdex::IN_ALGORITHM,    ustdex::let_value_t),    ustdex::WHAT (ustdex::FUNCTION_MUST_RETURN_A_SENDER),
ustdex::WITH_FUNCTION ((l
ambda at /home/eniebler/Code/ustdex/examples/scratch.cpp:64:25))>>'
[build] static_assert(_is_completion_signatures<completions>);
[build] ^~~~~~
[build] /home/eniebler/Code/ustdex/examples/scratch.cpp:74:12: note: in instantiation of function template specialization
'ustdex::sync_wait_t::operator
(<ustdex::start_on_t::_sndr_t<ustdex::run_loop::_scheduler,
ustdex::_let<ustdex::_value>::_sndr_t<ustdex::read_env_t::_sndr_t<ustdex::get_delegatee_sch
eduler_t>, (lambda at /home/eniebler/Code/ustdex/examples/scratch.cpp:64:25)>> &>' requested here
[build] sync_wait(s);
[build] ^
[build] 1 error generated.
```

This is the complete diagnostic. As you can see, the source of the error has been propagated out of the sender expression tree and reported at the API boundary, in `sync_wait`. The diagnostic contains only the information the user needs to fix the problem.

Another problem and a solution

For the authors of sender adaptor algorithms, this meta-error propagation technique presents a neigh insurmountable metaprogramming challenge. Computing completion signatures is hard enough. But now a child sender's completion signatures may not actually be completion signatures! They could instead be an error that the algorithm author must propagate in *their* completion signatures, or else lose the information about the root cause.

P2300 recognizes that manipulating completion signatures at compile time is taxing. It provides a utility to help: `transform_completion_signatures`. Given a set of completion signatures and some alias templates, it applies transformations to the set, resulting in a new set of completion signatures. Sender adaptor authors can use `transform_completion_signatures` to adapt the child sender's completions.

With a few small changes, `transform_completions_signatures` can be made to automatically propagate any intermediate types that might represent errors, saving users the trouble of doing so manually. This paper proposes those design changes.

Comparison Table

Consider the following use of `transform_completion_signatures`:

```
struct ERROR_TOO_MANY_VALUES {};  
  
template <class... Values>  
using _value_completions_t = std::conditional_t<  
    sizeof...(Values) > 1,  
    ERROR_TOO_MANY_VALUES,  
    completion_signatures<set_value_t(Values...)>  
>;  
  
// For a given sender and environment, check that the value completions  
// never send more than one value.  
template <class Sndr, class Env>  
using _checked_completions = transform_completion_signatures<  
    completion_signatures_of_t<Sndr, Env>,  
    completion_signatures<>,  
    _value_completions_t  
>;
```

Type computation	Result Before	Result After
<pre>using S = decltype(just(1, 2, 3)); using T = _checked_completions<S, empty_env>;</pre>	<i>hard error</i>	T is ERROR_TOO_MANY_VALUES
<pre>using S = decltype(read_env(get_scheduler)); using T = _checked_completions<S, empty_env>;</pre>	<i>hard error</i>	T is: ustdex::ERROR<ustdex::WHERE (ustdex::IN_ALGORITHM, ustdex::read, ustdex::WHAT (ustdex::THE_CURRENT_ENVIRONMENT_LACKS_THIS_QUERY), ::WITH_QUERY (ustdex::get_scheduler_t), ustdex::WITH_ENVIRONMENT x::env<>>) (with the ustdex library)

In the first case, the type error happens in the `_value_completions_t` alias template. In the second case, the type error happens when trying to ask the `read_env(get_scheduler)` sender what its completions are when used with `empty_env`. That's an error because the `empty_env` does not have a value for the `get_scheduler` query. In both cases, the error gets propagated by `transform_completion_signatures` after the proposed change.

Proposed Wording

[Editorial note: This proposed wording is based on [P2300R9](#). -- end note]

[Editorial note: Change [\[async.ops\]/13](#) as follows: -- end note]

- 13. A completion signature is a function type that describes a completion operation. An asynchronous operation has a finite set of possible completion signatures corresponding to the completion operations that the asynchronous operation potentially evaluates ([basic.def.odr]). For a completion function `set`, receiver `rcvr`, and pack of arguments `args`, let `c` be the completion operation `set(rcvr, args...)`, and let `F` be the function type `decltype(auto(set))(decltype((args))...)`. A completion signature `sig` is associated with `c` if and only if `MATCHING_SIG(sig, F)` is true ([exec.general]). Together, a sender type and an environment type `Env` determine the set of completion signatures of an asynchronous operation that results from connecting the sender with a receiver that has an environment of type `Env`. [Note: The type of the receiver does not affect an asynchronous operation's completion signatures, only the type of the receiver's environment. -- end note] A sender type whose completion signatures are knowable independent of an execution environment is known as a non-dependent sender.

[Editorial note: Change [\[exec.syn\]](#) as follows: -- end note]

```
...  
  
template<class Sndr, class... Env = empty_env> concept sender_in = see below; ...
```

template<class Sndr, class... Env ~~= empty_env~~> requires sender_in<Sndr, Env...> using completion_signatures_of_t = call_result_t<get_completion_signatures_t, Sndr, Env...>; ...

[Editorial note: Change [exec.snd.concepts] as follows: -- end note]

```
template<class Sndr, class... Env = empty_env>
concept sender_in =
    sender<Sndr> &&
    (sizeof...(Env) <= 1)
    (queryable<Env> &&...) &&
    requires (Sndr&& sndr, Env&&... env) {
        { get_completion_signatures(
            std::forward<Sndr>(sndr), std::forward<Env>(env)... ) }
        -> valid-completion-signatures;
    };
};
```

[Editorial note: this subtly changes the meaning of sender_in<Sndr>. Before the change, it tests whether a type is a sender when used specifically with the environment empty_env. After the change, it tests whether a type is a non-dependent sender. This is a stronger assertion to make about the type; it says that this type is a sender *regardless of the environment*. One can still get the old behavior with sender_in<Sndr, empty_env>. -- end note]

[Editorial note: Change [exec.awaitables] as follows: -- end note]

1. The sender concepts recognize awaitables as senders. For this clause ([exec]), an **awaitable** is an expression that would be well-formed as the operand of a co_await expression within a given context.
2. For a subexpression c, let GET-AWAITER(c, p) be expression-equivalent to the series of transformations and conversions applied to c as the operand of an *await-expression* in a coroutine, resulting in lvalue e as described by [expr.await]/3.2-4, where p is an lvalue referring to the coroutine's promise type, Promise. This includes the invocation of the promise type's await_transform member if any, the invocation of the operator co_await picked by overload resolution if any, and any necessary implicit conversions and materializations. Let GET-AWAITER(c) be expression-equivalent to GET-AWAITER(c, q) where q is an lvalue of an unspecified empty class type none-such that lacks an await transform member, and where coroutine_handle<none-such> behaves as coroutine_handle<void>.
3. Let is-awaitable be the following exposition-only concept:

```
template<class T>
concept await-suspend-result = see below;

template<class A, class... Promise>
concept is-awaiter = // exposition only
    requires (A& a, coroutine_handle<Promise...> h) {
        a.await_ready() ? 1 : 0;
        { a.await_suspend(h) } -> await-suspend-result;
        a.await_resume();
    };

template<class C, class... Promise>
concept is-awaitable =
    requires (C (*fc)() noexcept, Promise&... p) {
        { GET-AWAITER(fc(), p...) } -> is-awaiter<Promise...>;
    };
};
```

await-suspend-result<T> is true if and only if one of the following is true:

- o T is void, OR
- o T is bool, OR
- o T is a specialization of coroutine_handle.

4. For a subexpression c such that decltype((c)) is type c, and an lvalue p of type Promise, await-result-type<C, Promise> denotes the type decltype(GET-AWAITER(c, p).await_resume()) , and await-result-type<C> denotes the type decltype(GET-AWAITER(c).await_resume()).

[Editorial note: Change [exec.getcomplsigs] as follows: -- end note]

1. get_completion_signatures is a customization point object. Let sndr be an expression such that decltype((sndr)) is Sndr, and let env be ~~an expression such that decltype((env)) is Env~~ a pack of zero or one expressions. Then get_completion_aignatures(sndr, env...) is expression-equivalent to:
 1. remove_cvref_t<Sndr>::completion_signatures{} if that expression is well-formed,
 2. Otherwise, decltype(sndr.get_completion_signatures(env...)){} if that expression is well-formed,
 2. ~~Otherwise, remove_cvref_t<Sndr>::completion_signatures{} if that expression is well-formed,~~

3. Otherwise, if `is-awaitable<Sndr, env-promise<Env>...>` is true, then:

```
completion_signatures<
    SET-VALUE-SIG(await-result-type<Sndr, env-promise<Env>...>), // see
[exec.snd.concepts]
    set_error_t(exception_ptr),
    set_stopped_t()>{}
```

4. Otherwise, `get_completion_signatures(sndr, env...)` is ill-formed.

3. If `get_completion_signatures(sndr)` is well-formed and its type denotes a specialization of the `completion_signatures` class template, then `Sndr` is a non-dependent sender type ([`async.ops`]).

4. Given a type `Env`, if `completion_signatures_of_t<Sndr>` and `completion_signatures_of_t<Sndr, Env>` are both well-formed and denote instantiations of the `completion_signatures` class template, they shall denote the same set of completion signatures, with type equality determined with `MATCHING-SIG` ([`exec.general`]).

6. Let `rcvr` be an rvalue receiver of type `Rcvr...`.

[Editorial note: To [`exec.adapt.general`], add a paragraph (8) as follows: -- end note]

8. Unless otherwise specified, an adaptor whose child senders are all non-dependent ([`async.ops`]) is itself non-dependent. This requirement applies to any function that is selected by the implementation of the sender adaptor.

[Editorial note: Change [`exec.then`] as follows: -- end note]

2. The names `then`, `upon_error`, and `upon_stopped` denote customization point objects. For `then`, `upon_error`, and `upon_stopped`, let `set-cpo` be `set_value`, `set_error`, and `set_stopped` respectively. Let the expression `then-cpo` be one of `then`, `upon_error`, or `upon_stopped`. For subexpressions `sndr` and `f`, let `Sndr` be `decltype((sndr))` and let `F` be the decayed type of `f`. If `Sndr` does not satisfy sender, or `F` does not satisfy `movable-value`, `then-cpo(sndr, f)` is ill-formed.

3. Otherwise, let `invoke-result` be an alias template such that `invoke-result<Ts...>` denotes the type `invoke_result_t<F, Ts...>`. If `sender_in<Sndr>` is true and `gather-signatures<decayed-typeof<set-cpo>, completion_signatures_of_t<Sndr>, invoke-result, type-list>` is ill-formed, the program is ill-formed.

4. Otherwise, the expression `then-cpo(sndr, f)` is expression-equivalent to:.....

5. ~~For `then`, `upon_error`, and `upon_stopped`, let `set-cpo` be `set_value`, `set_error`, and `set_stopped` respectively.~~ The exposition-only class template `impls-for` ([`exec.snd.general`]) is specialized for `then-cpo` as follows:....

[Editorial note: Change [`exec.let`] by inserting a new paragraph between (4) and (5) as follows: -- end note]

5. Let `invoke-result` be an alias template such that `invoke-result<Ts...>` denotes the type `invoke_result_t<F, Ts...>`. If `sender_in<Sndr>` is true and `gather-signatures<decayed-typeof<set-cpo>, completion_signatures_of_t<Sndr>, invoke-result, type-list>` is ill-formed, or if any of the types in the resulting type list fail to satisfy sender, the program is ill-formed.

[Editorial note: Change [`exec.bulk`] by inserting a new paragraph between (3) and (4) as follows: -- end note]

4. Let `invoke-result` be an alias template such that `invoke-result<Ts...>` denotes the type `invoke_result_t<F, Shape, Ts...>`. If `sender_in<Sndr>` is true and `gather-signatures<decayed-typeof<set-cpo>, completion_signatures_of_t<Sndr>, invoke-result, type-list>` is ill-formed, the program is ill-formed.

[Editorial note: Change [`exec.split`] as follows: -- end note]

3. The names `split` and `ensure_started` denote customization point objects. Let the expression `shared-cpo` be one of `split` or `ensure_started`. For a subexpression `sndr`, let `Sndr` be `decltype((sndr))`. If `sender_in<Sndr, shared-env>` is false, ~~`shared-cpo(sndr)`~~ the program is ill-formed.

[Editorial note: Change [`exec.start.detached`] as follows: -- end note]

1. `start_detached` eagerly starts a sender without the caller needing to manage the lifetimes of any objects.

2. The name `start_detached` denotes a customization point object. For a subexpression `sndr`, let `Sndr` be `decltype((sndr))`. If `sender_in<Sndr, empty_env>` is false, ~~`start_detached`~~ the program is ill-formed. Otherwise ...

[Editorial note: Change [`exec.sync.wait`] as follows: -- end note]

4. The name `this_thread::sync_wait` denotes a customization point object. For a subexpression `sndr`, let `Sndr` be `decltype((sndr))`. If `sender_in<Sndr, sync_wait_env>` is false, the ~~expression `this_thread::sync_wait(sndr)`~~ the program is ill-formed. Otherwise, ~~it~~ the expression `this_thread::sync_wait(sndr)` is expression-equivalent to ...

[Editorial note: Change `[exec.utils.tfxcmplsig]` -- end note] as follows:

```
4. namespace std::execution {
    template<valid-completion-signaturesclass InputSignatures,
            valid-completion-signaturesclass AdditionalSignatures = completion_signatures<>,
            template<class...> class SetValue = default-set-value,
            template<class> class SetError = default-set-error,
            valid-completion-signaturesclass SetStopped = completion_signatures<set_stopped_t()>>
    using transform_completion_signatures = completion_signatures<see below>;
}
```

1. `SetValue` shall name an alias template such that for any template parameter pack `As...`, the type `SetValue<As...>` is either ill-formed or else ~~`valid-completion-signatures<SetValue<As...>>`~~ is satisfied.
2. `SetError` shall name an alias template such that for any type `Err`, `SetError<Err>` is either ill-formed or else ~~`valid-completion-signatures<SetError<Err>>`~~ is satisfied.

Then:

1. If ~~`valid-completion-signatures<E>`~~ is false where `E` is one of `InputSignatures`, `AdditionalSignatures`, OR `SetStopped`, `transform_completion_signatures<InputSignatures, AdditionalSignatures, SetValue, SetError, SetStopped>` denotes the type `E`. *[Note: If there are multiple types that fail to satisfy ~~`valid-completion-signatures`~~, it is unspecified which is chosen. -- end note]*
2. Let ~~`Vs...`~~ be a pack of the types in the *type-list* named by ~~`gather-signatures<set_value_t, InputSignatures, SetValue, type-list>`~~.
3. Let ~~`Es...`~~ be a pack of the types in the *type-list* named by ~~`gather-signatures<set_error_t, InputSignatures, type_identity_t, error-list>`~~, where *error-list* is an alias template such that ~~`error-list<Ts...>`~~ names ~~*type-list*~~`<SetError<Ts>...>`.
4. Let `Ss` name the type `completion_signatures<>` if ~~`gather-signatures<set_stopped_t, InputSignatures, type-list, type-list>`~~ is an alias for the type *type-list*; otherwise, `SetStopped`.

Then:

5. If any of the types in `Vs` or `Es` are ill-formed, then `transform_completion_signatures<InputSignatures, AdditionalSignatures, SetValue, SetError, SetStopped>` is ill-formed,
6. Otherwise, if any type `E` from set of types in `Vs` and `Es` fails to satisfy ~~`valid-completion-signatures`~~, then `transform_completion_signatures<InputSignatures, AdditionalSignatures, SetValue, SetError, SetStopped>` denotes the type `E`. *[Note: If more than one type in `Vs` and `Es` fail to satisfy ~~`valid-completion-signatures`~~, it is unspecified which is chosen. -- end note]*
7. Otherwise, `transform_completion_signatures<InputSignatures, AdditionalSignatures, SetValue, SetError, SetStopped>` names the type `completion_signatures<Sigs...>` where `Sigs...` is the unique set of types in all the template arguments of all the `completion_signatures` specializations in `[AdditionalSignatures, Vs..., Es..., Ss]`. For the purpose of uniqueness, type equality is determined with `MATCHING_SIG` (`[exec.general]`).

Acknowledgments

We owe our thanks to Ville Voutilainen who first noticed that most sender expressions could be type-checked eagerly but are not by P2300R8.